Middle East University
Faculty of Information Technology
Data Science and Artificial Intelligence Department
Course: Data Mining – Final Project

Project Title: AI-Assisted Public Threat Detection /Face Recognition System.

Team Members:

Name	Student ID
1.Hamzeh Al-Bawaneh	202311668
2. Lujain Al-Barghouthi	202311370
3.Donya Hani	202311141
4.Laylah Abaza	202311323

1. Project Idea and Objectives

1.1 Project Overview

Public safety has become increasingly dependent on intelligent surveillance systems capable of monitoring crowded environments and identifying potential threats before incidents occur. Traditional surveillance systems rely heavily on human operators, making continuous monitoring difficult, time-consuming, and prone to error.

The goal of this project is to develop an AI-Assisted Public Safety Threat Detection System capable of automatically detecting dangerous objects in real time using computer vision and deep learning techniques.

The proposed system focuses on detecting:

- Knives
- Pistols
- Bats

The system processes live video streams or surveillance footage and automatically identifies threatening objects. Once a weapon is detected, the system generates visual alerts and provides location information through bounding boxes around detected objects.

The project utilizes modern object detection techniques based on the YOLOv8 architecture, which is widely recognized for achieving high detection accuracy while maintaining real-time inference performance.

1.2 Project Objectives

The main objectives of this project are:

Primary Objectives

- Develop an intelligent weapon detection system.
- Detect and localize dangerous objects in real time.
- Achieve high detection accuracy while maintaining fast inference speed.
- Reduce false alarms and missed detection’s .
- Create a system suitable for future deployment in public safety environments.

Secondary Objectives

- Build a large-scale, balanced, and leakage-free dataset.
- Apply advanced preprocessing and cleaning techniques.
- Utilize transfer learning to improve performance.
- Evaluate model generalization using independent test data.
- Establish a foundation for future face recognition integration and intelligent monitoring systems.

---

2. Dataset Description and Preprocessing Techniques

2.1 Dataset Sources

The dataset was created by combining multiple publicly available weapon detection datasets collected from different sources. Images were selected to ensure diversity in:

- Background environments
- Lighting conditions
- Viewing angles
- Object sizes
- Image quality
- Camera perspectives

This diversity improves model robustness and generalization capability.

2.2 Final Dataset Statistics

The final dataset contains:

Metric	Value
Total Images	48,413
Total Objects	50,713

Class Distribution

Class	Objects
Knife	18,947
Pistol	14,597
Bat	17,169

Dataset Split

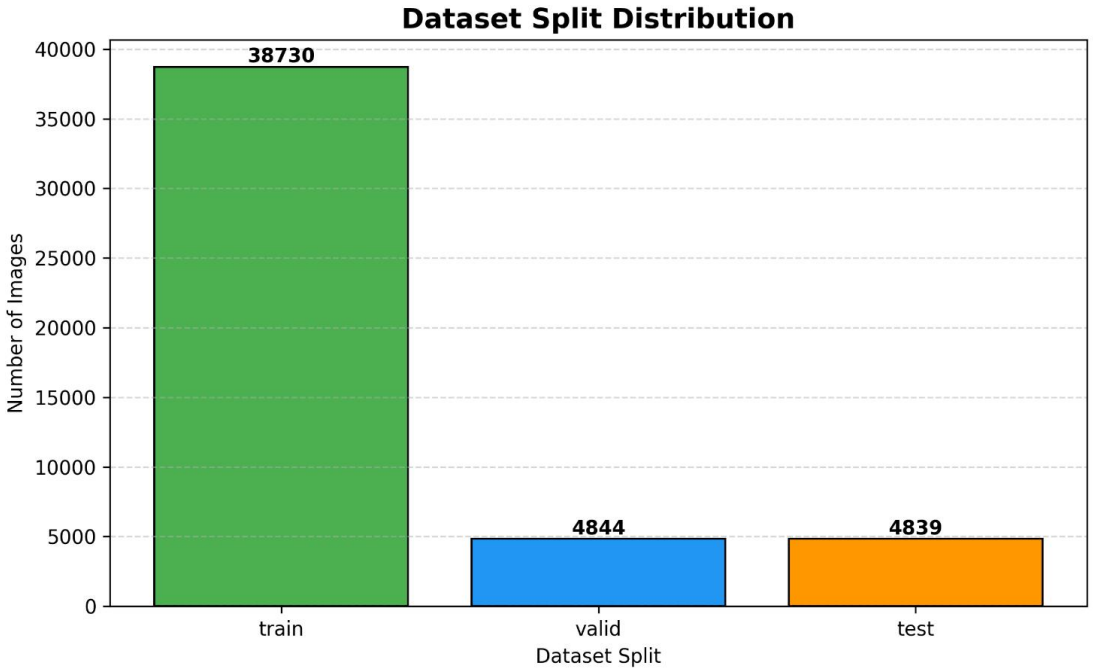

Figure 1: The dataset was organized according to the YOLO format and contains normalized bounding-box annotations for all weapon classes.

2.3 Data Preprocessing

Several preprocessing techniques were applied to improve dataset quality.

Annotation Verification

All annotation files were checked to identify:

- Missing labels
- Invalid bounding boxes
- Incorrect class IDs
- Corrupted annotation files

Invalid annotations were corrected or removed.

Duplicate Removal

Perceptual hashing techniques were applied to identify duplicate and near-duplicate images.

Benefits included:

- Reduced overfitting
- Improved generalization
- Prevention of repeated samples

Data Leakage Prevention

Duplicate-aware splitting was implemented before model training.

This ensured:

- No image duplication between train, validation, and test sets.
- Reliable performance evaluation.
- Accurate measurement of generalization capability.

Dataset Balancing

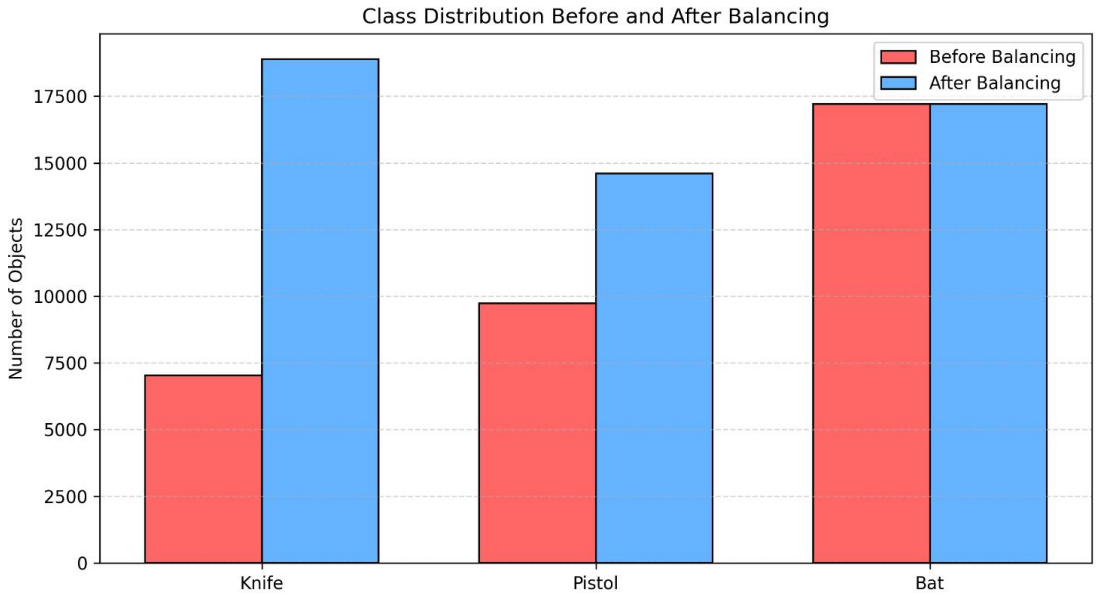

Figure 2: Class distributions were analyzed and balanced to reduce model bias toward any specific weapon category , by using oversampling techniques .

Image Normalization

Images were automatically resized and normalized according to YOLOv8 requirements.

---

3. Implementation and Methodology

3.1 Development Environment

The system was implemented using Python and modern deep learning libraries.

Component	Technology
Programming Language	Python 3.10
Deep Learning Framework	PyTorch
Object Detection Framework	Ultralytics YOLOv8
Computer Vision	OpenCV
Visualization	Matplotlib
Image Processing	Pillow
Development Platform	Jupyter Notebook

3.2 Hardware Configuration

Component	Specification
Processor	Intel Core i5 12th Gen
RAM	16 GB
GPU	NVIDIA RTX 3050 Laptop GPU
GPU Memory	4 GB
Operating System	Windows 11

3.3 Model Selection

Several object detection architectures were considered:

- R-CNN
- Fast R-CNN
- Faster R-CNN
- SSD

- YOLO

YOLOv8 was selected because it provides:

- High detection accuracy
- Real-time performance
- Efficient memory usage
- End-to-end training
- Fast inference speed

3.4 YOLOv8n Architecture

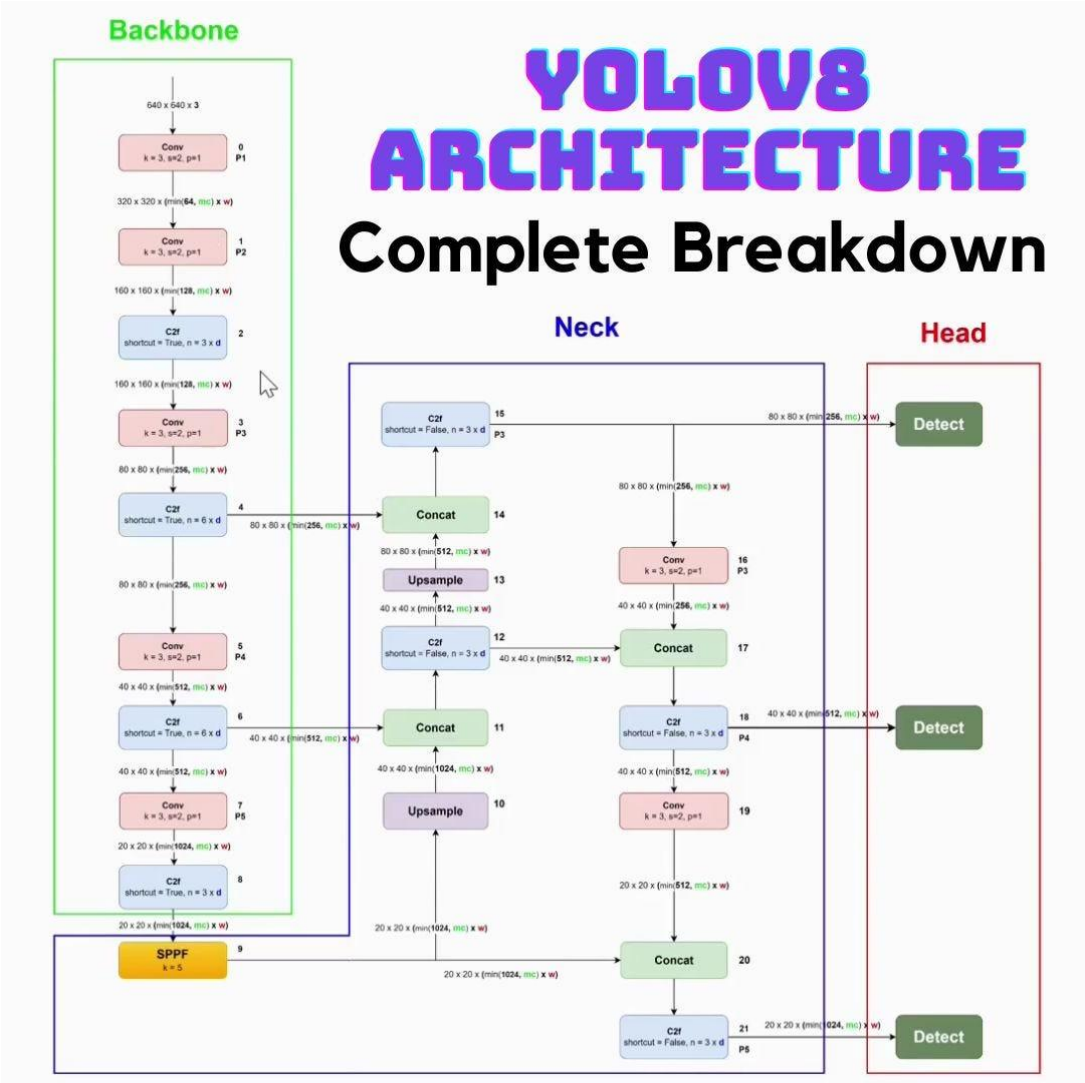

Figure 3: The selected model consists of three main components:

Backbone

Responsible for extracting image features such as:

- Edges
- Shapes
- Textures
- Object patterns

Neck

Combines features from multiple scales to improve detection of:

- Small objects
- Medium objects
- Large objects

Detection Head

Produces:

- Bounding boxes
- Class labels
- Confidence scores

3.5 Transfer Learning

Instead of training from scratch, transfer learning was applied using pretrained YOLOv8n weights trained on the COCO dataset.

Benefits included:

- Faster convergence
- Reduced training time
- Improved accuracy
- Better feature extraction

3.6 Training Configuration

Parameter	Value
Model	YOLOv8n
Epochs	100
Batch Size	16
Image Size	640×640
Workers	4
Device	CUDA GPU
Transfer Learning	Enabled

Training was performed for approximately **14.6 hours.**

3.7 Real-Time Threat Detection Pipeline

The final system follows the following workflow:

1. Capture video frames from camera.
2. Preprocess frame.
3. Run YOLOv8 inference.
4. Detect weapons.
5. Draw bounding boxes.
6. Calculate confidence scores.
7. Generate threat alerts.
8. Display results in real time.

This pipeline allows the system to operate continuously while maintaining real-time performance suitable for security applications.

3.8 System Architecture

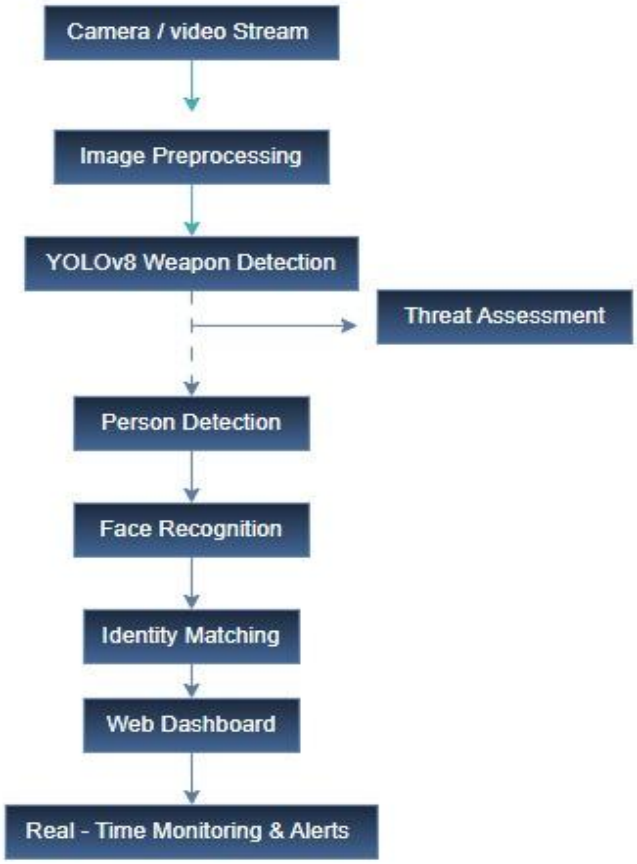

Figure 4: Complete System Architecture of the AI-Assisted Public Threat Detection System.

The system begins by capturing video frames from a live camera feed. Each frame undergoes preprocessing to improve image quality and visibility under different lighting conditions. The processed frame is then analyzed by the YOLOv8n object detection model to identify weapons such as knives, pistols, and bats. Person detection and face recognition modules operate alongside weapon detection to identify individuals present in the scene. The outputs of these modules are combined to assess the threat level and generate monitoring information. Finally, all detection results are displayed through the web-based dashboard, allowing security personnel to monitor events in real time.

4. Key Results and Metrics

4.1 Validation Results

Metric	Value
Precision	95.5%
Recall	92.6%
mAP50	96.6%
mAP50-95	78.1%

4.2 Test Results

Metric	Value
Precision	94.6%
Recall	92.5%
mAP50	96.5%
mAP50-95	77.7%

4.3 Class-Wise Results

Class	Precision	Recall	mAP50	mAP50-95
Knife	94.5%	91.0%	95.9%	75.1%
Pistol	93.9%	89.2%	94.7%	82.3%
Bat	95.4%	97.2%	98.7%	75.9%

4.4 Generalization Performance

The difference between validation and test metrics was very small:

Metric	Validation	Test
Precision	95.5%	94.6%
Recall	92.6%	92.5%
mAP50	96.6%	96.5%
mAP50-95	78.1%	77.7%

These results indicate strong generalization and minimal overfitting.

4.5 Confusion Matrix Analysis

The normalized confusion matrix was used to evaluate the classification performance of the YOLOv8n model across all weapon categories. The results indicate strong class separation and a low level of confusion between the three weapon classes.

Most knife, pistol, and bat instances were correctly classified, demonstrating that the model successfully learned discriminative visual features for each category. The pistol class showed slightly lower performance compared to the other classes due to variations in object size, viewing angles, and partial occlusions. However, the confusion levels remained low and did not significantly impact overall detection performance.

The confusion matrix confirms the reliability of the trained model and supports the high precision, recall, and mAP values obtained during validation and testing. These results indicate that the model is capable of accurately distinguishing between different weapon categories while maintaining strong generalization performance on unseen data.

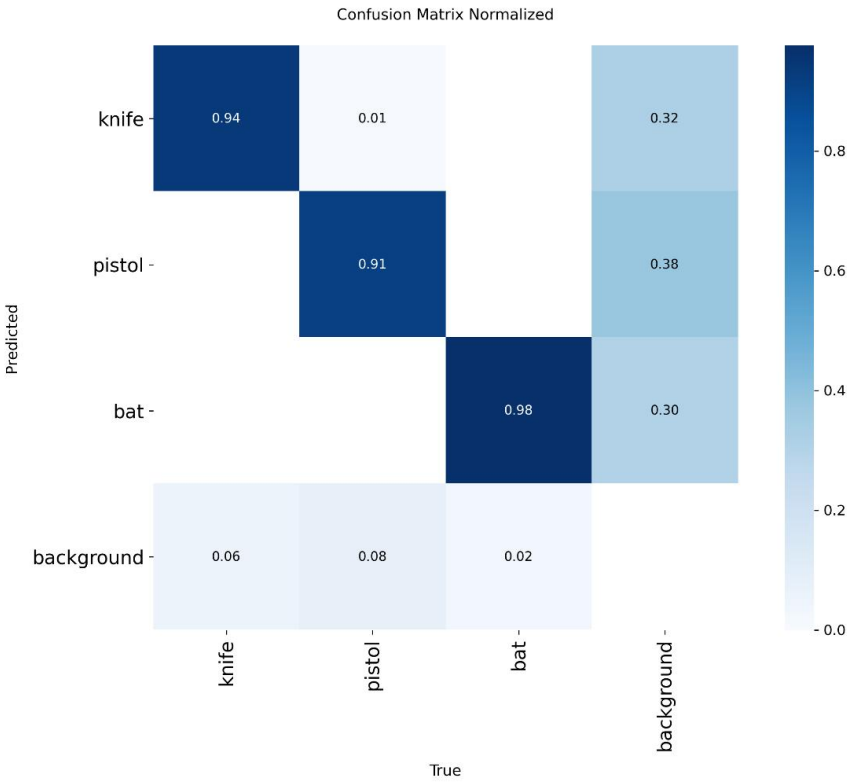

Figure 5: Normalized Confusion Matrix for the YOLOv8n Weapon Detection Model.

4.6 Training Performance Analysis

Training performance was monitored throughout the 100 training epochs using multiple evaluation metrics and loss functions provided by the YOLOv8 framework. The training curves demonstrate a stable learning process and successful convergence of the model.

The box loss, classification loss, and distribution focal loss (DFL) consistently decreased as training progressed, indicating that the model gradually improved its ability to localize and classify weapon objects. At the same time, precision, recall, mAP50, and mAP50-95 increased and stabilized toward the final training epochs.

The absence of significant fluctuations or divergence in the training curves suggests that the learning process remained stable throughout training. Furthermore, the close agreement between validation and test performance indicates that the model generalized effectively and did not suffer from severe overfitting.

These observations confirm that the selected training configuration, dataset preparation procedures, and transfer learning strategy contributed to the successful development of a robust weapon detection model.

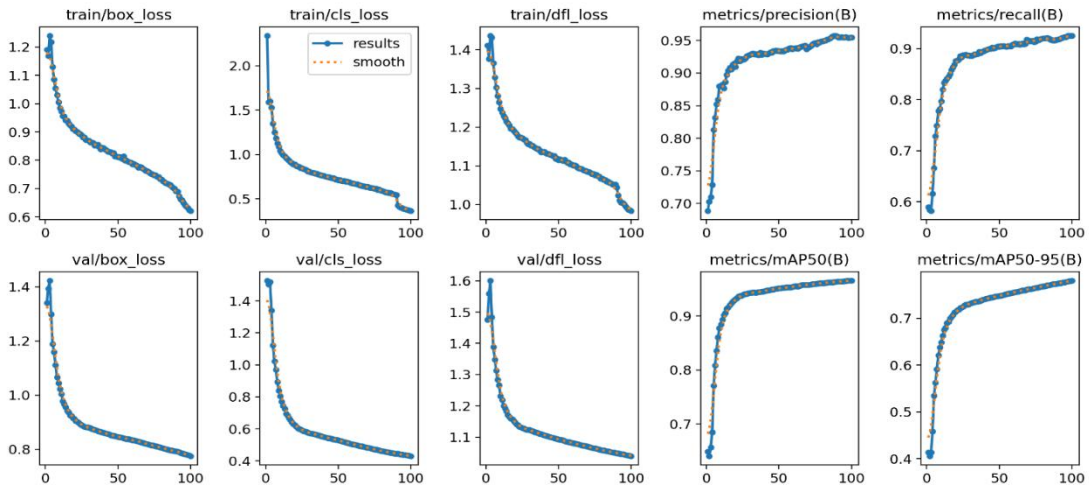

Figure 6: YOLOv8n Training Curves Showing Loss Reduction and Performance Improvement Throughout Training.

5. Web-based monitoring dashboard:

To provide a complete public safety monitoring solution, a web-based dashboard was developed and integrated with the AI detection system. The dashboard enables real-time monitoring of surveillance feeds and displays the output of the YOLOv8 weapon detection model through an intuitive user interface. Security personnel can start or stop camera monitoring directly from the dashboard and view live detection results, including detected weapon type, confidence score, threat level, and person identification status. The system dynamically updates detection information and highlights potential threats as they occur, allowing for rapid situational awareness and decision-making. The dashboard also serves as a foundation for future enhancements such as face recognition, event logging, alert notifications, and multi-camera monitoring. By combining real-time object detection with an interactive monitoring interface, the project moves beyond a standalone AI model and becomes a practical intelligent surveillance system suitable for public safety applications.

Figure 7: Web-Based Monitoring Dashboard Showing Live Camera Feed, Weapon Detection Results, Threat Assessment, Person Identification, and System Status Information.

6. Dashboard Components and Features

The monitoring dashboard serves as the primary user interface of the AI-Assisted Public Threat Detection System. It provides real-time visualization of detection results and allows users to interact with the surveillance system through an intuitive web-based environment.

The dashboard consists of the following components:

• **Live Camera Feed:** Displays the real-time video stream being analyzed by the AI system.

• **Weapon Detection Status:** Shows the currently detected weapon category, including knives, pistols, and bats.

• **Person Identification:** Displays the recognized individual when face recognition is successfully performed.

• **Confidence Score:** Presents the confidence level associated with the current detection result.

• **Threat Level Indicator:** Classifies detected events into predefined threat categories and highlights potentially dangerous situations.

• **Preprocessing Status:** Indicates whether image enhancement techniques have been applied to improve visibility under challenging lighting conditions.

• **Incident Management Functions:** Provides access to stored records, face database information, and data export functionality.

The dashboard continuously receives updates from the detection engine, allowing security personnel to monitor events as they occur. By combining live video analysis with real-time threat assessment, the dashboard transforms raw AI predictions into actionable security information.

6.1 Face Recognition Module

In addition to weapon detection, the system includes a face recognition module that helps identify known individuals in the surveillance scene. This module was added to support the project's goal of building a more complete AI-assisted public safety monitoring system.

The face recognition pipeline works as follows:

1. A person is detected in the video frame using the person-detection model.
2. The detected face region is extracted from the person crop.
3. Facial features are converted into a numerical embedding vector.
4. The generated embedding is compared against the stored face embeddings in the face database.
5. If the similarity score is above the matching threshold, the person is identified by name.
6. If no match is found, the system returns Unknown.

The face database is organized as a folder-based watchlist, where each person has a dedicated folder containing multiple face images. These images are used to generate reference embeddings for matching during runtime. This approach allows the system to recognize predefined individuals such as team members or authorized personnel.

The face recognition module is integrated with the web dashboard, where the detected person identity is displayed in real time together with the detected weapon type, confidence score, threat level, and preprocessing status. This integration makes the system more practical for real-time monitoring and security applications.

6.2 Threat Assessment Mechanism

To enhance situational awareness, the system includes a threat assessment mechanism that categorizes detected events into different threat levels. The threat level is determined based on the type of detected weapon and is displayed in real time through the monitoring dashboard.

The implemented threat categories are:

Detected Object	Threat Level
No Weapon Detected	NONE
Bat	MEDIUM
Knife	MEDIUM
Pistol	HIGH

When a weapon is detected, the corresponding threat level is automatically assigned and displayed on the dashboard together with the detected weapon type, confidence score, and person identification status. This allows security personnel to quickly evaluate the severity of a detected event without manually interpreting the detection results.

The threat assessment module serves as a decision-support component that transforms raw object detection outputs into meaningful security information, improving the usability of the overall surveillance system.

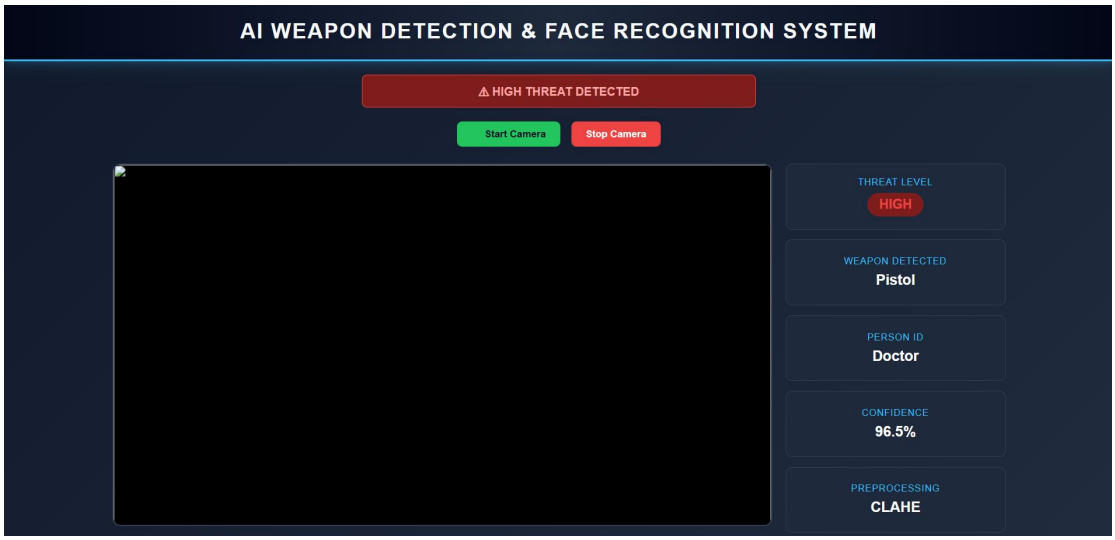

Figure 8: Threat Assessment and Real-Time Alert Display within the Monitoring Dashboard.

6.3 Face Database and Identity Management

To support face recognition functionality, a dedicated face database was developed and integrated into the monitoring system. The database stores images of known individuals organized into separate folders, where each folder represents a unique identity.

During system initialization, facial embeddings are generated from the stored images and loaded into memory. When a face is detected in a surveillance frame, a new embedding is generated and compared against the stored embeddings using a similarity-based matching process. If the similarity score exceeds a predefined threshold, the corresponding identity is returned. Otherwise, the individual is labeled as Unknown.

This watchlist-based approach allows the system to recognize predefined individuals such as authorized personnel, team members, or persons of interest. The modular structure of the face database enables new identities to be added without requiring retraining of the weapon detection model.

The face database is accessible through the web dashboard, allowing users to review registered identities and manage the face recognition component of the system. This functionality improves the practical applicability of the overall surveillance platform by combining weapon detection with identity awareness.

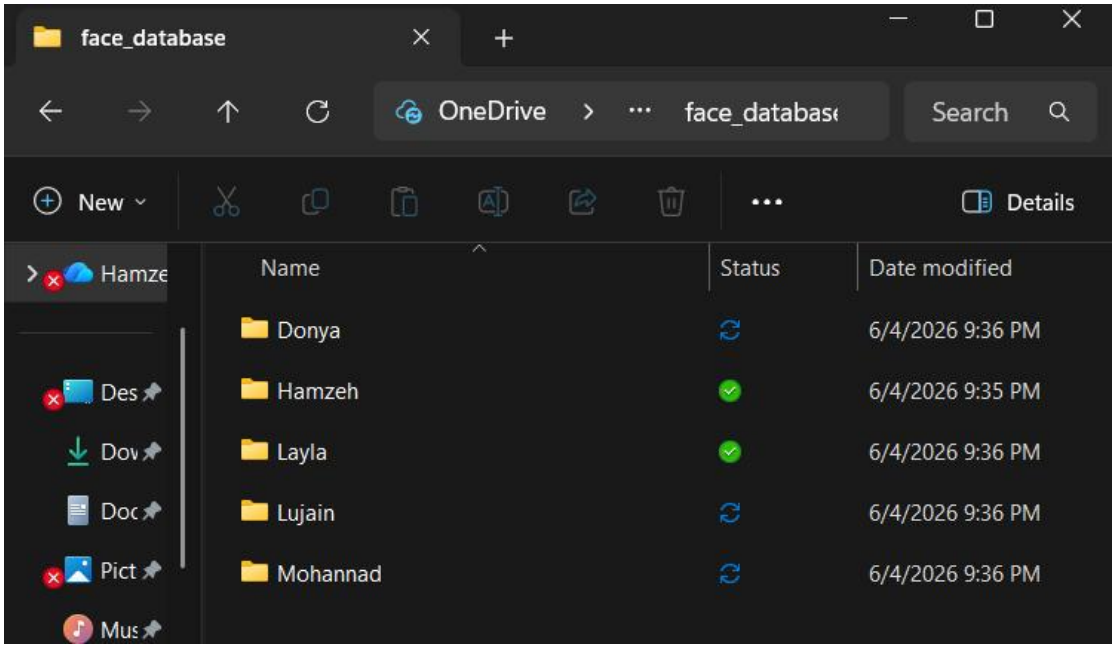

Figure 9: Face Database Interface Showing Registered Identities Used by the Face Recognition Module.

7. Discussion: What Worked, What Didn't, and Insights Learned

7.1 What Worked Well

Large and Diverse Dataset

The large dataset containing more than 48,000 images significantly improved model robustness and generalization.

Data Cleaning and Leakage Prevention

Removing duplicates and preventing leakage greatly improved evaluation reliability.

Transfer Learning

Using pretrained YOLOv8n weights accelerated convergence and improved final performance.

Real-Time Detection

YOLOv8n achieved excellent inference speed while maintaining high detection accuracy, making it suitable for surveillance applications.

Class Separation

The confusion matrix showed very limited confusion between weapon categories, indicating strong feature learning.

7.2 Challenges and Limitations

Pistol Detection

Pistols remained the most difficult class due to:

- Small object size
- Occlusions
- Similar backgrounds
- Variations in viewing angles

Real-World Deployment

The model was trained primarily on public datasets. Some real-world environments may contain conditions not fully represented during training.

Hardware Constraints

The RTX 3050 (4GB VRAM) limited experimentation with larger YOLOv8 models such as YOLOv8m and YOLOv8l.

Surveillance Integration

Full integration with enterprise surveillance systems was outside the scope of this project.

7.3 Lessons Learned

Several important lessons were learned throughout the project:

- Dataset quality is often more important than model complexity.
- Data leakage can produce misleading performance results.
- Transfer learning dramatically reduces training cost.
- Real-time systems require balancing accuracy and speed.
- Careful preprocessing significantly improves model performance.
- Robust evaluation requires independent test datasets.

---

8. Future Work

Future improvements may include:

- Multi-camera surveillance support.
- Video-based threat tracking.
- Mobile deployment.
- Alarm and notification systems.
- Cloud-based deployment.
- Training larger YOLOv8 variants.

- Integration with smart city infrastructure.

---

9. Conclusion

This project successfully developed an AI-Assisted Public Threat Detection System capable of detecting and localizing weapons in real time using the YOLOv8n object detection architecture. A large-scale custom dataset consisting of 48,413 images and 50,713 annotated objects was created through extensive collection, cleaning, balancing, duplicate removal, and data leakage prevention procedures. The final model achieved a Precision of 94.6%, Recall of 92.5%, mAP50 of 96.5%, and mAP50-95 of 77.7% on an independent test dataset, demonstrating strong detection accuracy and generalization capability.

Beyond weapon detection, the project integrated a face recognition module capable of identifying registered individuals through facial embedding comparison. The system also includes a threat assessment mechanism that classifies detected events according to their severity and presents the results through a real-time web-based monitoring dashboard.

The developed dashboard provides live camera monitoring, weapon detection status, person identification, confidence scores, threat-level visualization, and access to stored records and face database information. This integration transforms the project from a standalone object detection model into a practical intelligent surveillance platform suitable for public safety applications.

The project demonstrates that combining high-quality data engineering, transfer learning, computer vision, and web technologies can produce an effective AI-assisted monitoring solution. The developed system establishes a strong foundation for future enhancements such as multi-camera monitoring, advanced threat analytics, automated notifications, and large-scale deployment in smart security environments.

10. References

[1] Kaggle Weapon Detection Dataset : weapon-detection
[2] Roboflow Bat Dataset Computer Vision Model : Version
[3] Roboflow Pistols Computer Vision Dataset: Version
[4] Roboflow Knife detection dataset - Knife Detection (North) source 1: Knife detection - v1 2023-03-14 6:38pm
[5] Roboflow Knife Detection (COMSATS University Islamabad) - source 2: Knife Detection - v1 Knife-Detection
[6] Roboflow Knife Detection (Thesis) - source 3: Knife Detection - v4 2024-09-24 1:24pm
[7] Roboflow Knife Detection (SJZQP) - source 4: Knife Detection - v2 2023-06-02 1:27pm
[8] Roboflow Pistol Detection (Guardian RFID) - source 2: Pistol Detection Dataset > Overview

[9] Roboflow - Pistol Detection (Rajamangala Isan) - source 3: pistol - v9 2023-07-10 9:41pm
[10] Ultralytics YOLOv8 Documentation: Home | Ultralytics Docs
[11] OpenCV Documentation: OpenCV documentation index
[12] Face Recognition Library Documentation: GitHub - ageitgey/face_recognition: The world's simplest facial recognition api for Python and the command line · GitHub